

# Sales Forecasting Project

## Milestone 1: Data Collection, Exploration, and Preprocessing

### 1. Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
```

### 2. Data Collection

```
train=pd.read_csv("store-sales-time-series-forecasting/train.csv",parse_dates=['date'])
stores=pd.read_csv("store-sales-time-series-forecasting/stores.csv")
oil=pd.read_csv("store-sales-time-series-forecasting/oil.csv",parse_dates=['date'])
holidays_events=pd.read_csv("store-sales-time-series-forecasting/holidays_events.csv",parse_dates=['date'])
transactions = pd.read_csv("store-sales-time-series-forecasting/transactions.csv", parse_dates=['date'])
```

### 3. Initial Data Preview

```
train.head()
```

```
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000888 entries, 0 to 3000887
Data columns (total 6 columns):
 #   Column      Dtype
---  -
 0   id          int64
 1   date        datetime64[ns]
 2   store_nbr   int64
 3   family      object
 4   sales       float64
 5   onpromotion int64
dtypes: datetime64[ns](1), float64(1), int64(3), object(1)
memory usage: 137.4+ MB
```

```
train.describe()
```

```
stores.head()
```

```
stores.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 54 entries, 0 to 53
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    store_nbr   54 non-null    int64
1    city        54 non-null    object
2    state       54 non-null    object
3    type        54 non-null    object
4    cluster     54 non-null    int64
dtypes: int64(2), object(3)
memory usage: 2.2+ KB

```

```
stores.describe()
```

```
oil.head()
```

```
oil.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1218 entries, 0 to 1217
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    date        1218 non-null  datetime64[ns]
1    dcoilwtico  1175 non-null  float64
dtypes: datetime64[ns](1), float64(1)
memory usage: 19.2 KB

```

```
oil.describe()
```

```
holidays_events.head()
```

```
holidays_events.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 350 entries, 0 to 349
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    date        350 non-null  datetime64[ns]
1    type        350 non-null  object
2    locale      350 non-null  object
3    locale_name  350 non-null  object
4    description  350 non-null  object
5    transferred  350 non-null  bool
dtypes: bool(1), datetime64[ns](1), object(4)
memory usage: 14.1+ KB

```

```
holidays_events.describe()
```

```
transactions.head()
```

```
transactions.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 83488 entries, 0 to 83487
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   date             83488 non-null  datetime64[ns]
1   store_nbr        83488 non-null  int64
2   transactions     83488 non-null  int64
dtypes: datetime64[ns](1), int64(2)
memory usage: 1.9 MB
```

```
transactions.describe()
```

## 4. Missing Values Check

```
train.isnull().sum()
```

```
stores.isnull().sum()
```

```
oil.isnull().sum()
```

```
holidays_events.isnull().sum()
```

```
transactions.isnull().sum()
```

## 5. Duplicates Check

```
train.duplicated().sum()
```

```
stores.duplicated().sum()
```

```
oil.duplicated().sum()
```

```
holidays_events.duplicated().sum()
```

```
transactions.duplicated().sum()
```

## 6. Missing Values Handling

```
oil = oil.sort_values("date")
oil["dcoilwtico"] = oil["dcoilwtico"].ffill().bfill()
```

```
oil.isnull().sum()
```

## 7. Date Range Checks

```
train["date"].min(), "to", train["date"].max()
```

```
oil["date"].min(), "to", oil["date"].max()
```

```
transactions["date"].min(), "to", transactions["date"].max()
```

```
holidays_events["date"].min(), "to", holidays_events["date"].max()
```

## 8. Key Duplicates Check

```
train.duplicated(subset=["date", "store_nbr", "family"]).sum()
```

```
stores.duplicated(subset=["store_nbr"]).sum()
```

```
oil.duplicated(subset=["date"]).sum()
```

```
holidays_events.duplicated(subset=["date"]).sum()
```

```
transactions.duplicated(subset=["date", "store_nbr"]).sum()
```

## 9. Feature Engineering

```
train["quarter"] = train["date"].dt.quarter
train["dayofyear"] = train["date"].dt.dayofyear
train["is_month_start"] = train["date"].dt.is_month_start.astype(int)
train["is_month_end"] = train["date"].dt.is_month_end.astype(int)
train["is_quarter_start"] = train["date"].dt.is_quarter_start.astype(int)
train["is_quarter_end"] = train["date"].dt.is_quarter_end.astype(int)
train["is_year_start"] = train["date"].dt.is_year_start.astype(int)
train["is_year_end"] = train["date"].dt.is_year_end.astype(int)
```

```
train["month"] = train["date"].dt.month
train["dayofweek"] = train["date"].dt.dayofweek
```

```
train["month_sin"] = np.sin(2*np.pi*train["month"]/12)
train["month_cos"] = np.cos(2*np.pi*train["month"]/12)
```

```
train["dow_sin"] = np.sin(2*np.pi*train["dayofweek"]/7)
train["dow_cos"] = np.cos(2*np.pi*train["dayofweek"]/7)
train["month_cos"] = np.cos(2*np.pi*train["month"]/12)
```

```
train["dow_sin"] = np.sin(2*np.pi*train["dayofweek"]/7)
```

```

train["dow_cos"] = np.cos(2*np.pi*train["dayofweek"]/7)

train = train.sort_values(["store_nbr", "family", "date"])

train["lag_1"] = train.groupby(
    ["store_nbr", "family"]
)["sales"].shift(1)

train["lag_7"] = train.groupby(
    ["store_nbr", "family"]
)["sales"].shift(7)

train["lag_14"] = train.groupby(
    ["store_nbr", "family"]
)["sales"].shift(14)

train["lag_28"] = train.groupby(
    ["store_nbr", "family"]
)["sales"].shift(28)

train["rolling7"] = (
    train.groupby(["store_nbr", "family"])["sales"]
    .transform(lambda x:x.shift(1).rolling(7).mean())
)

train["rolling30"] = (
    train.groupby(["store_nbr", "family"])["sales"]
    .transform(lambda x:x.shift(1).rolling(30).mean())
)

train["promo_binary"]=(train["onpromotion"]>0).astype(int)

train["promo_log"]=np.log1p(train["onpromotion"])

stores["cluster"] = stores["cluster"].astype("category")
stores["type"] = stores["type"].astype("category")
stores["city"] = stores["city"].astype("category")
stores["state"] = stores["state"].astype("category")

train["family_code"] = train["family"].astype("category").cat.codes
stores["city_code"] = stores["city"].cat.codes
stores["state_code"] = stores["state"].cat.codes
stores["type_code"] = stores["type"].cat.codes
stores["cluster_code"] = stores["cluster"].cat.codes

train[["family", "family_code"]].drop_duplicates().head()

oil["oil_change"]=oil["dcoilwtico"].diff()

oil["oil_pct_change"]=oil["dcoilwtico"].pct_change()

oil["oil_roll7"]=oil["dcoilwtico"].rolling(7).mean()

oil["oil_roll30"]=oil["dcoilwtico"].rolling(30).mean()

```

```
holiday_count = holidays_events.groupby("date").size()
```

```
transactions["trans_lag1"] = transactions.groupby(
    "store_nbr"
)["transactions"].shift(1)
```

```
transactions["trans_roll7"] = (
    transactions.groupby("store_nbr")["transactions"]
    .transform(lambda x: x.shift(1).rolling(7).mean())
)
```

```
transactions["trans_change"] = transactions.groupby(
    "store_nbr")["transactions"].pct_change()
```

## 10. Dataset Merging

```
df = train.copy()
```

```
df = df.merge(
    stores,
    on="store_nbr",
    how="left"
)
```

```
df = df.merge(
    transactions,
    on=["store_nbr", "date"],
    how="left"
)
```

```
df = df.merge(
    oil,
    on="date",
    how="left"
)
```

```
df["dcoilwtico"] = df["dcoilwtico"].ffill().bfill()
df["transactions"] = df["transactions"].fillna(0)
```

```
print(df.shape)
df.head()
```

```
(3000888, 46)
```

```
df["date"] = pd.to_datetime(df["date"])
```

```
df["day_of_week"] = df["date"].dt.dayofweek
df["is_weekend"] = df["day_of_week"].isin([5, 6]).astype(int)
```

```
if "promo_binary" not in df.columns:
    df["promo_binary"] = (df["onpromotion"] > 0).astype(int)
```

```
if "promo_log" not in df.columns:
    df["promo_log"] = np.log1p(df["onpromotion"])
```

```
df["promo_weekend"] = df["promo_binary"] * df["is_weekend"]
df["oil_transactions"] = df["dcoilwtico"] * df["transactions"]

df[["date", "day_of_week", "is_weekend", "promo_binary", "promo_weekend", "oil_transactions"]
```

## 11. Holiday Features

```
holidays_events["type"].value_counts()
holidays_events["locale"].value_counts()
holidays_events["transferred"].value_counts()

hol = holidays_events.copy()

# keep only non-transferred rows
hol = hol[hol["transferred"] == False].copy()

# split holidays and events
hol_holiday = hol[hol["type"] == "Holiday"].copy()
hol_event = hol[hol["type"] == "Event"].copy()

# national holidays
national_holidays = hol_holiday[hol_holiday["locale"] == "National"][["date"]].drop_duplicates()
national_holidays["is_national_holiday"] = 1

# regional holidays
regional_holidays = hol_holiday[hol_holiday["locale"] == "Regional"][["date", "locale_name"]].drop_duplicates()
regional_holidays = regional_holidays.rename(columns={"locale_name": "state"})
regional_holidays["is_regional_holiday"] = 1

# local holidays
local_holidays = hol_holiday[hol_holiday["locale"] == "Local"][["date", "locale_name"]].drop_duplicates()
local_holidays = local_holidays.rename(columns={"locale_name": "city"})
local_holidays["is_local_holiday"] = 1

# events by date
events_by_date = hol_event[["date"]].drop_duplicates()
events_by_date["is_event"] = 1

# rebuild final df
df = train.merge(stores, on="store_nbr", how="left")
df = df.merge(transactions, on=["store_nbr", "date"], how="left")
df = df.merge(oil, on="date", how="left")

# merge holiday features
df = df.merge(national_holidays, on="date", how="left")
df = df.merge(regional_holidays, on=["date", "state"], how="left")
df = df.merge(local_holidays, on=["date", "city"], how="left")
df = df.merge(events_by_date, on="date", how="left")

# fill missing values after merging
df["dcoilwtico"] = df["dcoilwtico"].ffill().bfill()
df["transactions"] = df["transactions"].fillna(0)

# fill holiday flags
df["is_national_holiday"] = df["is_national_holiday"].fillna(0).astype(int)
df["is_regional_holiday"] = df["is_regional_holiday"].fillna(0).astype(int)
df["is_local_holiday"] = df["is_local_holiday"].fillna(0).astype(int)
```

```

df["is_event"] = df["is_event"].fillna(0).astype(int)

# combined holiday flag
df["is_holiday"] = (
    (df["is_national_holiday"] == 1) |
    (df["is_regional_holiday"] == 1) |
    (df["is_local_holiday"] == 1)
).astype(int)

print(df.shape)
df[["transactions", "dcoilwtico", "is_national_holiday", "is_regional_holiday", "is_local_holiday"]]

(3000888, 51)

```

## 12. Time-Based Features

```

df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["day"] = df["date"].dt.day
df["dayofweek"] = df["date"].dt.dayofweek
df["weekofyear"] = df["date"].dt.isocalendar().week.astype(int)
df["is_weekend"] = (df["dayofweek"] >= 5).astype(int)

df["promo_binary"] = (df["onpromotion"] > 0).astype(int)
df["promo_log"] = np.log1p(df["onpromotion"])
df["promo_weekend"] = df["promo_binary"] * df["is_weekend"]
df["oil_transactions"] = df["dcoilwtico"] * df["transactions"]

df[["year", "month", "day", "dayofweek", "weekofyear", "is_weekend", "promo_weekend", "oil_transactions"]]

```

## 13. Summary Statistics and Outlier Detection

```

print(df["sales"].describe())
print("Skewness:", df["sales"].skew())
print("Zero Sales Ratio:", (df["sales"] == 0).mean())

```

```

count      3.000888e+06
mean       3.577757e+02
std        1.101998e+03
min        0.000000e+00
25%        0.000000e+00
50%        1.100000e+01
75%        1.958473e+02
max        1.247170e+05
Name: sales, dtype: float64
Skewness: 7.3587578188826335
Zero Sales Ratio: 0.3129506999261552

```

```

Q1 = df["sales"].quantile(0.25)
Q3 = df["sales"].quantile(0.75)
IQR = Q3 - Q1

```

```

lower_bound = Q1 - 1.5 * IQR

```

```
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["sales"] < lower_bound) | (df["sales"] > upper_bound)]
print("Number of potential outliers:", outliers.shape[0])

Number of potential outliers: 447105
```

## 14. Exploratory Data Analysis Visualizations

```
daily_sales = df.groupby("date")["sales"].sum().reset_index()
```

```
fig = px.line(
    daily_sales,
    x="date",
    y="sales",
    title="Daily Total Sales Over Time",
    labels={
        "date": "Date",
        "sales": "Total Sales"
    }
)
```

```
fig.update_layout(
    xaxis_title="Date",
    yaxis_title="Total Sales",
    hovermode="x unified"
)
```

```
fig.show()
```

```
monthly_sales = df.groupby(["year", "month"])["sales"].mean().reset_index()
```

```
monthly_sales["year_month"] = (
    monthly_sales["year"].astype(str) + "-" + monthly_sales["month"].astype(str).str.zfill(2)
)
```

```
fig = px.line(
    monthly_sales,
    x="year_month",
    y="sales",
    title="Average Sales by Year-Month",
    labels={
        "year_month": "Year-Month",
        "sales": "Average Sales"
    },
    markers=True
)
```

```
fig.update_layout(
    xaxis_title="Year-Month",
    yaxis_title="Average Sales",
    hovermode="x unified"
)
```

```
fig.update_xaxes(
    tickangle=90,
    rangelider_visible=True
)
```

```

)

fig.show()

dow_sales = df.groupby("dayofweek")["sales"].mean().reset_index()

dow_sales["day_name"] = dow_sales["dayofweek"].map({
    0: "Monday",
    1: "Tuesday",
    2: "Wednesday",
    3: "Thursday",
    4: "Friday",
    5: "Saturday",
    6: "Sunday"
})

fig = px.bar(
    dow_sales,
    x="day_name",
    y="sales",
    title="Average Sales by Day of Week",
    labels={
        "day_name": "Day of Week",
        "sales": "Average Sales"
    },
    text="sales"
)

fig.update_traces(
    texttemplate="%{text:.2f}",
    textposition="outside"
)

fig.update_layout(
    xaxis_title="Day of Week",
    yaxis_title="Average Sales",
    hovermode="x unified"
)

fig.show()

holiday_effect = df.groupby("is_holiday")["sales"].mean().reset_index()

holiday_effect["Holiday Status"] = holiday_effect["is_holiday"].map({
    0: "Non-Holiday",
    1: "Holiday"
})

fig = px.bar(
    holiday_effect,
    x="Holiday Status",
    y="sales",
    title="Average Sales: Holiday vs Non-Holiday",
    labels={
        "Holiday Status": "Holiday Status",
        "sales": "Average Sales"
    },
    text="sales"
)

```

```

fig.update_traces(
    texttemplate="%{text:.2f}",
    textposition="outside"
)

fig.update_layout(
    xaxis_title="Holiday Status",
    yaxis_title="Average Sales",
    hovermode="x unified"
)

fig.show()

df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["day"] = df["date"].dt.day
df["dayofweek"] = df["date"].dt.dayofweek
df["weekofyear"] = df["date"].dt.isocalendar().week.astype(int)
df["is_weekend"] = (df["dayofweek"] >= 5).astype(int)

month_sales = df.groupby("month")["sales"].mean().reset_index()

month_sales["month_name"] = month_sales["month"].map({
    1: "January",
    2: "February",
    3: "March",
    4: "April",
    5: "May",
    6: "June",
    7: "July",
    8: "August",
    9: "September",
    10: "October",
    11: "November",
    12: "December"
})

fig = px.bar(
    month_sales,
    x="month_name",
    y="sales",
    title="Average Sales by Month",
    labels={
        "month_name": "Month",
        "sales": "Average Sales"
    },
    text="sales"
)

fig.update_traces(
    texttemplate="%{text:.2f}",
    textposition="outside"
)

fig.update_layout(
    xaxis_title="Month",
    yaxis_title="Average Sales",
    hovermode="x unified"
)

```

```

fig.show()

daily_sales_plot = daily_sales.copy()

if "date" not in daily_sales_plot.columns:
    daily_sales_plot = daily_sales_plot.reset_index()

daily_sales_plot["rolling_30"] = daily_sales_plot["sales"].rolling(30).mean()

fig = px.line(
    daily_sales_plot,
    x="date",
    y=["sales", "rolling_30"],
    title="Daily Sales with 30-Day Rolling Mean",
    labels={
        "date": "Date",
        "value": "Sales",
        "variable": "Metric"
    }
)

fig.update_layout(
    xaxis_title="Date",
    yaxis_title="Sales",
    hovermode="x unified"
)

fig.update_xaxes(
    rangeslider_visible=True
)

fig.show()

```

## 15. Promotion, Transactions, and External Factors Analysis

```

df["has_promo"] = (df["onpromotion"] > 0).astype(int)
df.groupby("has_promo")["sales"].mean()

import plotly.express as px

promo_sales = df.groupby("onpromotion")["sales"].mean().reset_index()

fig = px.scatter(
    promo_sales,
    x="onpromotion",
    y="sales",
    title="Sales vs Onpromotion",
    labels={
        "onpromotion": "Onpromotion",
        "sales": "Average Sales"
    },
    hover_data=["onpromotion", "sales"]
)

fig.update_layout(
    xaxis_title="Onpromotion",
    yaxis_title="Average Sales",
    hovermode="closest"
)

```

```

)

fig.show()

df_trans = df.copy()
print(df_trans.shape)
print(df_trans[["sales", "transactions"]].corr())

(3000888, 58)

```

|              | sales    | transactions |
|--------------|----------|--------------|
| sales        | 1.000000 | 0.233081     |
| transactions | 0.233081 | 1.000000     |

```

sample_data = df_trans[["sales", "transactions"]].dropna().sample(5000, random_state=42)

fig = px.scatter(
    sample_data,
    x="transactions",
    y="sales",
    title="Sales vs Transactions",
    labels={
        "transactions": "Transactions",
        "sales": "Sales"
    },
    opacity=0.5,
    hover_data=["transactions", "sales"]
)

fig.update_layout(
    xaxis_title="Transactions",
    yaxis_title="Sales",
    hovermode="closest"
)

fig.show()

store_trans = df_trans.groupby("store_nbr")[["sales", "transactions"]].mean().reset_index()
store_trans.head()

df[["sales", "dcoilwtico"]].corr()

oil_daily = df.groupby("date")["dcoilwtico"].mean().reset_index()

fig = px.line(
    oil_daily,
    x="date",
    y="dcoilwtico",
    title="Oil Price Over Time",
    labels={
        "date": "Date",
        "dcoilwtico": "Oil Price"
    }
)

fig.update_layout(

```

```

        xaxis_title="Date",
        yaxis_title="Oil Price",
        hovermode="x unified"
    )

fig.update_xaxes(
    rangeslider_visible=True
)

fig.show()

df.groupby("is_holiday")["sales"].mean()
df.groupby("is_event")["sales"].mean()

```

## 16. Store and Product Analysis

```

import plotly.express as px

store_type_sales = df.groupby("type")["sales"].mean().reset_index()

fig = px.bar(
    store_type_sales,
    x="type",
    y="sales",
    title="Average Sales by Store Type",
    labels={
        "type": "Store Type",
        "sales": "Average Sales"
    },
    text="sales"
)

fig.update_traces(
    texttemplate="%{text:.2f}",
    textposition="outside"
)

fig.update_layout(
    xaxis_title="Store Type",
    yaxis_title="Average Sales",
    hovermode="x unified"
)

fig.show()

```

C:\Users\Fatoma\AppData\Local\Temp\ipykernel\_18296\4126360634.py:3: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version

```

cluster_sales = df.groupby("cluster")["sales"].mean().reset_index()

fig = px.bar(
    cluster_sales,
    x="cluster",
    y="sales",
    title="Average Sales by Cluster",

```

```

        labels={
            "cluster": "Cluster",
            "sales": "Average Sales"
        },
        text="sales"
    )

```

```

fig.update_traces(
    texttemplate="%{text:.2f}",
    textposition="outside"
)

```

```

fig.update_layout(
    xaxis_title="Cluster",
    yaxis_title="Average Sales",
    hovermode="x unified"
)

```

```

fig.show()

```

C:\Users\Fatoma\AppData\Local\Temp\ipykernel\_18296\1286893471.py:1: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version

```

top_stores = (
    df.groupby("store_nbr")["sales"]
    .mean()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

```

```

top_stores["store_nbr"] = top_stores["store_nbr"].astype(str)

```

```

fig = px.bar(
    top_stores,
    x="store_nbr",
    y="sales",
    title="Top 10 Stores by Average Sales",
    labels={
        "store_nbr": "Store Number",
        "sales": "Average Sales"
    },
    text="sales"
)

```

```

fig.update_traces(
    texttemplate="%{text:.2f}",
    textposition="outside"
)

```

```

fig.update_layout(
    xaxis_title="Store Number",
    yaxis_title="Average Sales",
    hovermode="x unified"
)

```

```

fig.show()

```

```

top_families = (
    df.groupby("family")["sales"]
      .mean()
      .sort_values(ascending=False)
      .head(10)
      .reset_index()
)

fig = px.bar(
    top_families,
    x="family",
    y="sales",
    title="Top 10 Product Families by Average Sales",
    labels={
        "family": "Product Family",
        "sales": "Average Sales"
    },
    text="sales"
)

fig.update_traces(
    texttemplate="%{text:.2f}",
    textposition="outside"
)

fig.update_layout(
    xaxis_title="Product Family",
    yaxis_title="Average Sales",
    hovermode="x unified"
)

fig.update_xaxes(
    tickangle=45
)

fig.show()

```

```

top_family_names = (
    df.groupby("family")["sales"]
      .mean()
      .sort_values(ascending=False)
      .head(5)
      .index
)

family_time = (
    df[df["family"].isin(top_family_names)]
      .groupby(["date", "family"])["sales"]
      .mean()
      .reset_index()
)

fig = px.line(
    family_time,
    x="date",
    y="sales",
    color="family",
    title="Sales Over Time for Top Families",
    labels={
        "date": "Date",
        "sales": "Average Sales",
    }
)

```

```

        "family": "Product Family"
    }
)

fig.update_layout(
    xaxis_title="Date",
    yaxis_title="Average Sales",
    hovermode="x unified"
)

fig.update_xaxes(
    rangeslider_visible=True
)

fig.show()

```

## 17. Correlation Heatmap

```

candidate_cols = [
    "sales",
    "onpromotion",
    "dcoilwtico",
    "year",
    "month",
    "day",
    "dayofweek",
    "weekofyear",
    "is_weekend",
    "is_national_holiday",
    "is_regional_holiday",
    "is_local_holiday",
    "is_event",
    "is_holiday",
    "is_transferred"
]

num_cols = [col for col in candidate_cols if col in df.columns]

corr_matrix = df[num_cols].corr().round(2)

fig = px.imshow(
    corr_matrix,
    text_auto=True,
    title="Correlation Heatmap",
    labels=dict(color="Correlation"),
    color_continuous_scale="RdBu",
    zmin=-1,
    zmax=1
)

fig.update_layout(
    width=1000,
    height=800
)

fig.show()

```

# Milestone 2: Advanced Data Analysis and Feature Engineering

```
import warnings
warnings.filterwarnings("ignore")

import matplotlib.pyplot as plt
import seaborn as sns

from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
import plotly.graph_objects as go
```

## 1. Time Series Analysis

```
# Create Daily Sales Time Series
daily_sales = df.groupby("date")["sales"].sum().reset_index()

daily_sales = daily_sales.set_index("date")

print(daily_sales.head())
plt.figure(figsize=(15,6))

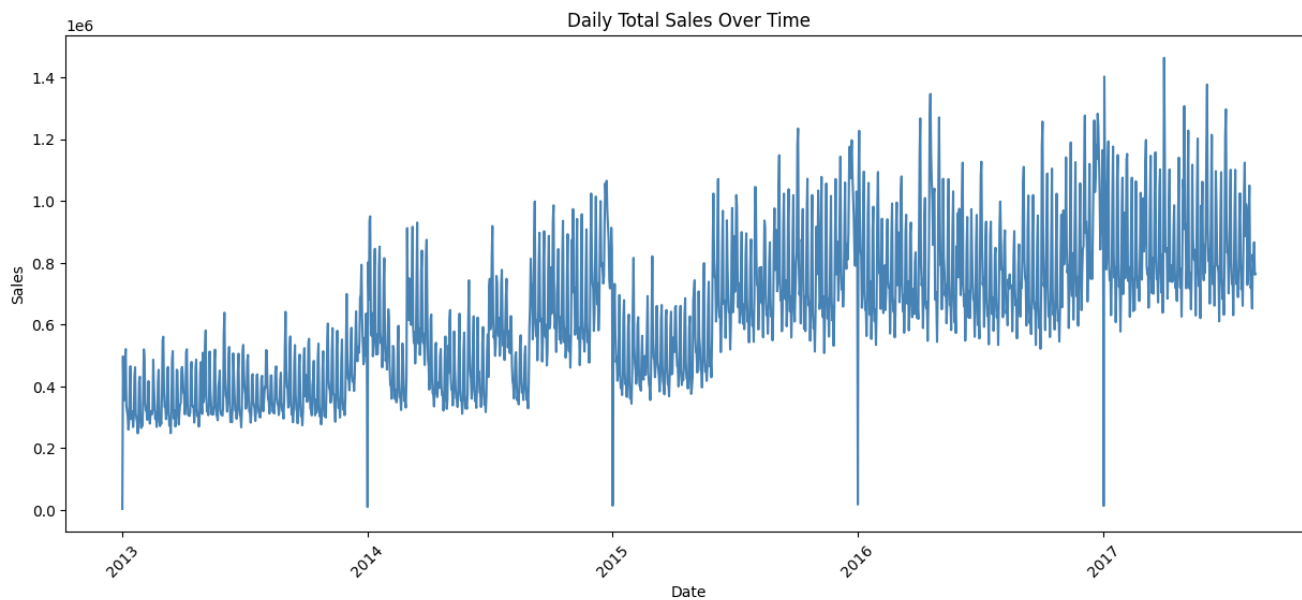
plt.plot(
    daily_sales.index,
    daily_sales["sales"],
    color="steelblue"
)

plt.title("Daily Total Sales Over Time")
plt.xlabel("Date")
plt.ylabel("Sales")

plt.xticks(rotation=45)

plt.show()
```

|            | sales         |
|------------|---------------|
| date       |               |
| 2013-01-01 | 2511.618999   |
| 2013-01-02 | 496092.417944 |
| 2013-01-03 | 361461.231124 |
| 2013-01-04 | 354459.677093 |
| 2013-01-05 | 477350.121229 |



```
result = seasonal_decompose(
    daily_sales["sales"],
    model="additive",
    period=365
)
```

```
result.plot()
```

```
plt.show()
```



```

daily_sales_plot = daily_sales.copy()

if "date" not in daily_sales_plot.columns:
    daily_sales_plot = daily_sales_plot.reset_index()

daily_sales_plot["rolling_30"] = (
    daily_sales_plot["sales"]
    .rolling(window=30, min_periods=1)
    .mean()
)

fig = px.line(
    daily_sales_plot,
    x="date",
    y=["sales", "rolling_30"],
    title="Daily Sales with 30-Day Rolling Mean",
    labels={
        "date": "Date",
        "value": "Sales",
        "variable": "Metric"
    }
)

fig.update_layout(
    xaxis_title="Date",
    yaxis_title="Sales",
    hovermode="x unified"
)

fig.update_xaxes(
    rangeslider_visible=True
)

fig.show()

```

## 2. Stationarity Testing

```

# ADF TEST

result = adfuller(daily_sales["sales"])

print("===== ORIGINAL SERIES =====")

print("ADF Statistic :", result[0])

print("p-value :", result[1])

if result[1] < 0.05:

    print("Series is Stationary")

else:
    print("Series is Non-Stationary")

# First Order Differencing

daily_sales["sales_diff"] = daily_sales["sales"].diff()

daily_sales_diff = daily_sales.dropna().copy()

```

```
===== ORIGINAL SERIES =====  
ADF Statistic : -2.6161957486048575  
p-value : 0.08969592175787455  
Series is Non-Stationary
```

```
# ADF Test After Differencing  
result2 = adfuller(daily_sales_diff["sales_diff"])  
  
print("\n===== AFTER DIFFERENCING =====")  
  
print("ADF Statistic :", result2[0])  
  
print("p-value :", result2[1])  
  
if result2[1] < 0.05:  
    print("Series is Stationary after Differencing")  
else:  
    print("Series is Still Non-Stationary")
```

```
===== AFTER DIFFERENCING =====  
ADF Statistic : -11.497599762661025  
p-value : 4.573768417343224e-21  
Series is Stationary after Differencing
```

```
# Plot Differenced Series  
  
plt.figure(figsize=(15,5))  
  
plt.plot(  
    daily_sales_diff.index,  
    daily_sales_diff["sales_diff"],  
    color="darkorange"  
)  
  
plt.title("Differenced Sales Series")  
  
plt.xlabel("Date")  
  
plt.ylabel("Differenced Sales")  
  
plt.show()
```



### 3. Correlation Analysis

```
# Sales vs Promotion
corr_promo = df["sales"].corr(df["onpromotion"])

print("Sales vs Promotion Correlation:", corr_promo)
```

Sales vs Promotion Correlation: 0.42792320481213353

```
# Sales vs Oil Price
corr_oil = df["sales"].corr(df["dcoilwtico"])

print("Sales vs Oil Price Correlation:", corr_oil)
```

Sales vs Oil Price Correlation: -0.0748083795654158

```
# Sales vs Transactions
corr_transactions = df_trans["sales"].corr(df_trans["transactions"])

print("Sales vs Transactions Correlation:", corr_transactions)
```

Sales vs Transactions Correlation: 0.23308113453485438

```
# Sales vs Holidays
holiday_sales = df.groupby("is_holiday")["sales"].mean()

print("\nAverage Sales")

print("Normal Days :", holiday_sales[0])

print("Holiday Days:", holiday_sales[1])
```

Average Sales  
 Normal Days : 358.01610400219863  
 Holiday Days: 349.0399593455964

## 4. Advanced Visualizations

```
top_families = (
    df.groupby("family")["sales"]
      .mean()
      .sort_values(ascending=False)
      .head(5)
      .index
)

family_daily = (
    df[df["family"].isin(top_families)]
      .groupby(["date", "family"])["sales"]
      .mean()
      .reset_index()
)

fig = go.Figure()

for i, family in enumerate(top_families):
    temp = family_daily[family_daily["family"] == family]

    fig.add_trace(
        go.Scatter(
            x=temp["date"],
            y=temp["sales"],
            mode="lines",
            name=family,
            visible=(i == 0)
        )
    )

buttons = []

for i, family in enumerate(top_families):
    visible = [False] * len(top_families)
    visible[i] = True

    buttons.append(
        dict(
            label=family,
            method="update",
            args=[
                {"visible": visible},
                {"title": f"Sales Trend Dashboard - {family}"}
            ]
        )
    )

fig.update_layout(
    title=f"Sales Trend Dashboard - {top_families[0]}",
    xaxis_title="Date",
    yaxis_title="Average Sales",
    updatemenus=[
        dict(
            buttons=buttons,
            direction="down",
            showactive=True
        )
    ]
)
```

```

    ]
)

fig.update_xaxes(rangeslider_visible=True)

fig.show()

if "has_promo" not in df.columns:
    df["has_promo"] = (df["onpromotion"] > 0).astype(int)

df["Promotion Status"] = df["has_promo"].map({
    0: "No Promotion",
    1: "Promotion"
})

fig = px.box(
    df,
    x="Promotion Status",
    y="sales",
    title="Interactive Promotion Impact on Sales",
    labels={
        "Promotion Status": "Promotion",
        "sales": "Sales"
    },
    points="outliers"
)

fig.update_yaxes(
    range=[0, df["sales"].quantile(0.99)]
)

fig.update_layout(
    xaxis_title="Promotion",
    yaxis_title="Sales"
)

fig.show()

# Holiday Impact
holiday_effect = df.groupby("is_holiday")["sales"].mean().reset_index()

holiday_effect["Holiday Status"] = holiday_effect["is_holiday"].map({
    0: "Non-Holiday",
    1: "Holiday"
})

fig = px.bar(
    holiday_effect,
    x="Holiday Status",
    y="sales",
    title="Interactive Average Sales: Holiday vs Non-Holiday",
    labels={
        "Holiday Status": "Holiday",
        "sales": "Average Sales"
    },
    text="sales"
)

fig.update_traces(
    texttemplate="%{text:.2f}",

```

```

        textposition="outside"
    )

fig.update_layout(
    xaxis_title="Holiday",
    yaxis_title="Average Sales",
    hovermode="x unified"
)

fig.show()

```

## 5. Final Cleaned Dataset Export

```

numeric_cols = df.select_dtypes(include=[np.number]).columns
category_cols = df.select_dtypes(include=["category"]).columns

# Convert categorical columns to object before filling new text values
for col in category_cols:
    df[col] = df[col].astype("object")

object_cols = df.select_dtypes(include=["object"]).columns

df[numeric_cols] = df[numeric_cols].replace([np.inf, -np.inf], np.nan)
df[numeric_cols] = df[numeric_cols].fillna(0)
df[object_cols] = df[object_cols].fillna("Unknown")

print("Missing values:", df.isnull().sum().sum())
print("Duplicate keys:", df.duplicated(subset=["date", "store_nbr", "family"]).sum())
print("Final shape:", df.shape)

df.to_csv("clean_store_sales.csv", index=False)

Missing values: 0
Duplicate keys: 0
Final shape: (3000888, 59)

```

## Milestone 3: Machine Learning Model Development and Optimization

### XGBoost Regressor

```

from xgboost import XGBRegressor

from sklearn.model_selection import TimeSeriesSplit, cross_val_score
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

import numpy as np
import pandas as pd
import plotly.express as px
import joblib

```

```

model_df = df.copy()

model_df["date"] = pd.to_datetime(model_df["date"])
model_df = model_df.sort_values("date")

model_df = model_df.dropna(subset=["sales"])

target = "sales"

numeric_features = [
    "store_nbr",
    "onpromotion",
    "dcoilwtico",
    "transactions",
    "year",
    "month",
    "day",
    "dayofweek",
    "weekofyear",
    "is_weekend",
    "is_holiday",
    "is_national_holiday",
    "is_regional_holiday",
    "is_local_holiday",
    "is_event",
    "cluster"
]

categorical_features = [
    "family",
    "city",
    "state",
    "type"
]

numeric_features = [col for col in numeric_features if col in model_df.columns]
categorical_features = [col for col in categorical_features if col in model_df.columns]

features = numeric_features + categorical_features

X = model_df[features]
y = model_df[target]

split_index = int(len(model_df) * 0.8)

X_train = X.iloc[:split_index]
X_test = X.iloc[split_index:]

y_train = y.iloc[:split_index]
y_test = y.iloc[split_index:]

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)

print("Train dates:", model_df["date"].iloc[:split_index].min(), "to", model_df["date"].iloc[split_index:].min())
print("Test dates:", model_df["date"].iloc[split_index:].min(), "to", model_df["date"].iloc[split_index:].max())

Training shape: (2400710, 20)
Testing shape: (600178, 20)
Train dates: 2013-01-01 00:00:00 to 2016-09-12 00:00:00

```

Test dates: 2016-09-12 00:00:00 to 2017-08-15 00:00:00

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)  
    ],  
    remainder="passthrough"  
)
```

```
xgb_model = XGBRegressor(  
    n_estimators=300,  
    learning_rate=0.05,  
    max_depth=8,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    objective="reg:squarederror",  
    random_state=42,  
    n_jobs=-1  
)
```

```
xgb_pipeline = Pipeline(  
    steps=[  
        ("preprocessor", preprocessor),  
        ("model", xgb_model)  
    ]  
)
```

```
tscv = TimeSeriesSplit(n_splits=3)
```

```
cv_scores = cross_val_score(  
    xgb_pipeline,  
    X_train,  
    y_train,  
    cv=tscv,  
    scoring="neg_root_mean_squared_error",  
    n_jobs=-1  
)
```

```
rmse_scores = -cv_scores
```

```
print("Cross-validation RMSE scores:", rmse_scores)  
print("Mean CV RMSE:", rmse_scores.mean())  
print("Standard Deviation:", rmse_scores.std())
```

```
Cross-validation RMSE scores: [484.150065  397.29613581 543.92713909]  
Mean CV RMSE: 475.12444663552  
Standard Deviation: 60.20110254155746
```

```
xgb_pipeline.fit(X_train, y_train)
```

```
y_pred = xgb_pipeline.predict(X_test)
```

```
y_pred = np.maximum(y_pred, 0)
```

```
xgb_mae = mean_absolute_error(y_test, y_pred)
```

```

xgb_mse = mean_squared_error(y_test, y_pred)
xgb_rmse = np.sqrt(xgb_mse)
xgb_r2 = r2_score(y_test, y_pred)

xgb_results = pd.DataFrame({
    "Model": ["XGBoost Regressor"],
    "MAE": [xgb_mae],
    "MSE": [xgb_mse],
    "RMSE": [xgb_rmse],
    "R2 Score": [xgb_r2]
})

xgb_results

results_df = pd.DataFrame({
    "date": model_df["date"].iloc[split_index:].values,
    "actual_sales": y_test.values,
    "predicted_sales": y_pred
})

daily_results = results_df.groupby("date", as_index=False).agg({
    "actual_sales": "sum",
    "predicted_sales": "sum"
})

fig = px.line(
    daily_results,
    x="date",
    y=["actual_sales", "predicted_sales"],
    title="Actual vs Predicted Sales - XGBoost",
    labels={
        "date": "Date",
        "value": "Sales",
        "variable": "Sales Type"
    }
)

fig.update_layout(
    hovermode="x unified"
)

fig.update_xaxes(
    rangeslider_visible=True
)

fig.show()

residuals = y_test.values - y_pred

residual_df = pd.DataFrame({
    "predicted_sales": y_pred,
    "residuals": residuals
})

sample_residuals = residual_df.sample(
    min(5000, len(residual_df)),
    random_state=42
)

fig = px.scatter(

```

```

    sample_residuals,
    x="predicted_sales",
    y="residuals",
    title="Residual Plot - XGBoost",
    labels={
        "predicted_sales": "Predicted Sales",
        "residuals": "Residuals"
    },
    opacity=0.5
)

fig.add_hline(y=0, line_dash="dash")

fig.show()

```

## SARIMA Model

```

from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
import pandas as pd
import plotly.express as px

sarima_df = df.copy()

sarima_df["date"] = pd.to_datetime(sarima_df["date"])

daily_sales_sarima = (
    sarima_df.groupby("date")["sales"]
    .sum()
    .reset_index()
)

daily_sales_sarima = daily_sales_sarima.sort_values("date")

daily_sales_sarima = daily_sales_sarima.set_index("date")

# Make sure the time series has daily frequency
daily_sales_sarima = daily_sales_sarima.asfreq("D")

# Fill missing dates if any
daily_sales_sarima["sales"] = daily_sales_sarima["sales"].ffill().bfill()

daily_sales_sarima.head()

split_index = int(len(daily_sales_sarima) * 0.8)

train_sarima = daily_sales_sarima.iloc[:split_index]
test_sarima = daily_sales_sarima.iloc[split_index:]

print("Train period:", train_sarima.index.min(), "to", train_sarima.index.max())
print("Test period:", test_sarima.index.min(), "to", test_sarima.index.max())
print("Train shape:", train_sarima.shape)
print("Test shape:", test_sarima.shape)

Train period: 2013-01-01 00:00:00 to 2016-09-11 00:00:00
Test period: 2016-09-12 00:00:00 to 2017-08-15 00:00:00

```

```
Train shape: (1350, 1)
Test shape: (338, 1)
```

```
sarima_model = SARIMAX(
    train_sarima["sales"],
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 7),
    enforce_stationarity=False,
    enforce_invertibility=False
)
```

```
sarima_result = sarima_model.fit(dispatch=False)
```

```
print(sarima_result.summary())
```

```

SARIMAX Results
=====
Dep. Variable:          sales      No. Observations:          1350
Model:          SARIMAX(1, 1, 1)x(1, 1, 1, 7)      Log Likelihood          -17111.19
Date:          Mon, 06 Jul 2026      AIC          34232.38
Time:          17:16:02      BIC          34258.35
Sample:          01-01-2013      HQIC          34242.11
              - 09-11-2016
Covariance Type:          opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          0.4615      0.039      11.735      0.000      0.384      0.539
ma.L1         -0.8729      0.035     -24.697      0.000     -0.942     -0.804
ar.S.L7        -0.0235      0.052      -0.450      0.652     -0.126      0.079
ma.S.L7        -0.9276      0.023     -40.071      0.000     -0.973     -0.882
sigma2       1.371e+10     1.49e-12     9.19e+21      0.000     1.37e+10     1.37e+10
=====
Ljung-Box (L1) (Q):          0.02      Jarque-Bera (JB):          38052.79
Prob(Q):          0.88      Prob(JB):          0.00
Heteroskedasticity (H):          2.09      Skew:          -0.88
Prob(H) (two-sided):          0.00      Kurtosis:          29.12
=====
```

```
Warnings:
```

```
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 4.28e+36. Stars
```

```
sarima_forecast = sarima_result.forecast(steps=len(test_sarima))
```

```
sarima_pred = np.maximum(sarima_forecast.values, 0)
```

```
y_test_sarima = test_sarima["sales"].values
```

```
sarima_mae = mean_absolute_error(y_test_sarima, sarima_pred)
```

```
sarima_mse = mean_squared_error(y_test_sarima, sarima_pred)
```

```
sarima_rmse = np.sqrt(sarima_mse)
```

```
sarima_r2 = r2_score(y_test_sarima, sarima_pred)
```

```
sarima_results = pd.DataFrame({
    "Model": ["SARIMA"],
    "MAE": [sarima_mae],
    "MSE": [sarima_mse],
```

```

    "RMSE": [sarima_rmse],
    "R2 Score": [sarima_r2]
})

```

```

sarima_results

```

```

sarima_plot_df = pd.DataFrame({
    "date": test_sarima.index,
    "actual_sales": y_test_sarima,
    "predicted_sales": sarima_pred
})

```

```

fig = px.line(
    sarima_plot_df,
    x="date",
    y=["actual_sales", "predicted_sales"],
    title="Actual vs Predicted Sales - SARIMA",
    labels={
        "date": "Date",
        "value": "Sales",
        "variable": "Sales Type"
    }
)

```

```

fig.update_layout(
    hovermode="x unified"
)

```

```

fig.update_xaxes(
    rangeslider_visible=True
)

```

```

fig.show()

```

```

sarima_residuals = y_test_sarima - sarima_pred

```

```

sarima_residual_df = pd.DataFrame({
    "predicted_sales": sarima_pred,
    "residuals": sarima_residuals
})

```

```

fig = px.scatter(
    sarima_residual_df,
    x="predicted_sales",
    y="residuals",
    title="Residual Plot - SARIMA",
    labels={
        "predicted_sales": "Predicted Sales",
        "residuals": "Residuals"
    },
    opacity=0.6
)

```

```

fig.add_hline(y=0, line_dash="dash")

```

```

fig.show()

```

```

from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

```

```

import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX

series = daily_sales_sarima["sales"]

n_splits = 3
test_size = 30 # each validation fold = 30 days

cv_results = []

for i in range(n_splits):
    train_end = len(series) - test_size * (n_splits - i)
    test_end = train_end + test_size

    train_cv = series.iloc[:train_end]
    valid_cv = series.iloc[train_end:test_end]

    model_cv = SARIMAX(
        train_cv,
        order=(1, 1, 1),
        seasonal_order=(1, 1, 1, 7),
        enforce_stationarity=False,
        enforce_invertibility=False
    )

    result_cv = model_cv.fit(dispatch=False)

    forecast_cv = result_cv.forecast(steps=len(valid_cv))
    forecast_cv = np.maximum(forecast_cv.values, 0)

    mae_cv = mean_absolute_error(valid_cv.values, forecast_cv)
    mse_cv = mean_squared_error(valid_cv.values, forecast_cv)
    rmse_cv = np.sqrt(mse_cv)

    cv_results.append({
        "Fold": i + 1,
        "Train Start": train_cv.index.min(),
        "Train End": train_cv.index.max(),
        "Validation Start": valid_cv.index.min(),
        "Validation End": valid_cv.index.max(),
        "MAE": mae_cv,
        "MSE": mse_cv,
        "RMSE": rmse_cv
    })

sarima_cv_results = pd.DataFrame(cv_results)

sarima_cv_results

sarima_cv_summary = pd.DataFrame({
    "Model": ["SARIMA Cross Validation"],
    "Mean MAE": [sarima_cv_results["MAE"].mean()],
    "Mean MSE": [sarima_cv_results["MSE"].mean()],
    "Mean RMSE": [sarima_cv_results["RMSE"].mean()],
    "STD RMSE": [sarima_cv_results["RMSE"].std()]
})

sarima_cv_summary

```

## Linear Regression Baseline Model

```
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import TimeSeriesSplit, cross_val_score
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
import numpy as np
import pandas as pd
import plotly.express as px
```

```
lr_model = LinearRegression()
```

```
lr_pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("model", lr_model)
    ]
)
```

```
tscv = TimeSeriesSplit(n_splits=3)
```

```
lr_cv_scores = cross_val_score(
    lr_pipeline,
    X_train,
    y_train,
    cv=tscv,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1
)
```

```
lr_rmse_scores = -lr_cv_scores
```

```
print("Linear Regression Cross-validation RMSE scores:", lr_rmse_scores)
print("Mean CV RMSE:", lr_rmse_scores.mean())
print("Standard Deviation:", lr_rmse_scores.std())
```

```
Linear Regression Cross-validation RMSE scores: [693.82513772 731.22101224 841.76615456]
Mean CV RMSE: 755.6041015083282
Standard Deviation: 62.80943346285957
```

```
lr_pipeline.fit(X_train, y_train)
```

```
lr_pred = lr_pipeline.predict(X_test)
```

```
# Sales cannot be negative
lr_pred = np.maximum(lr_pred, 0)
```

```
lr_mae = mean_absolute_error(y_test, lr_pred)
lr_mse = mean_squared_error(y_test, lr_pred)
lr_rmse = np.sqrt(lr_mse)
lr_r2 = r2_score(y_test, lr_pred)
```

```
lr_results = pd.DataFrame({
    "Model": ["Linear Regression"],
    "MAE": [lr_mae],
```

```

    "MSE": [lr_mse],
    "RMSE": [lr_rmse],
    "R2 Score": [lr_r2],
    "CV Mean RMSE": [lr_rmse_scores.mean()],
    "CV STD RMSE": [lr_rmse_scores.std()]
})

lr_results

lr_results_df = X_test.copy()

lr_results_df["date"] = model_df.loc[X_test.index, "date"].values
lr_results_df["actual_sales"] = y_test.values
lr_results_df["predicted_sales"] = lr_pred

lr_daily_results = lr_results_df.groupby("date", as_index=False).agg({
    "actual_sales": "sum",
    "predicted_sales": "sum"
})

fig = px.line(
    lr_daily_results,
    x="date",
    y=["actual_sales", "predicted_sales"],
    title="Actual vs Predicted Sales - Linear Regression",
    labels={
        "date": "Date",
        "value": "Sales",
        "variable": "Sales Type"
    }
)

fig.update_layout(
    hovermode="x unified"
)

fig.update_xaxes(
    rangeslider_visible=True
)

fig.show()

lr_residuals = y_test.values - lr_pred

lr_residual_df = pd.DataFrame({
    "predicted_sales": lr_pred,
    "residuals": lr_residuals
})

lr_sample_residuals = lr_residual_df.sample(
    min(5000, len(lr_residual_df)),
    random_state=42
)

fig = px.scatter(
    lr_sample_residuals,
    x="predicted_sales",
    y="residuals",
    title="Residual Plot - Linear Regression",
    labels={

```

```

        "predicted_sales": "Predicted Sales",
        "residuals": "Residuals"
    },
    opacity=0.5
)

fig.add_hline(y=0, line_dash="dash")

fig.show()

```

## Tuned Random Forest Regressor

```

from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import TimeSeriesSplit, cross_val_score, RandomizedSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import numpy as np
import pandas as pd
import plotly.express as px
import joblib

rf_base_model = RandomForestRegressor(
    random_state=42,
    n_jobs=-1
)

rf_base_pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("model", rf_base_model)
    ]
)

tscv = TimeSeriesSplit(n_splits=2)

param_grid = {
    "model__n_estimators": [10, 20],
    "model__max_depth": [6, 10],
    "model__min_samples_split": [10],
    "model__min_samples_leaf": [5]
}

rf_random_search = RandomizedSearchCV(
    estimator=rf_base_pipeline,
    param_distributions=param_grid,
    n_iter=3,
    cv=tscv,
    scoring="neg_root_mean_squared_error",
    random_state=42,
    n_jobs=-1,
    verbose=1
)

rf_random_search.fit(X_train, y_train)

print("Best Parameters:")
print(rf_random_search.best_params_)

```

```

print("Best CV RMSE:")
print(-rf_random_search.best_score_)

    Fitting 2 folds for each of 3 candidates, totalling 6 fits
    Best Parameters:
    {'model__n_estimators': 20, 'model__min_samples_split': 10, 'model__min_samples_leaf': 5}
    Best CV RMSE:
    513.491146343281

best_rf_pipeline = rf_random_search.best_estimator_

rf_pred = best_rf_pipeline.predict(X_test)

# Sales cannot be negative
rf_pred = np.maximum(rf_pred, 0)

rf_mae = mean_absolute_error(y_test, rf_pred)
rf_mse = mean_squared_error(y_test, rf_pred)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_pred)

rf_results = pd.DataFrame({
    "Model": ["Tuned Random Forest Regressor"],
    "MAE": [rf_mae],
    "MSE": [rf_mse],
    "RMSE": [rf_rmse],
    "R2 Score": [rf_r2],
    "Best CV RMSE": [-rf_random_search.best_score_]
})

rf_results

rf_results_df = X_test.copy()

rf_results_df["date"] = model_df.loc[X_test.index, "date"].values
rf_results_df["actual_sales"] = y_test.values
rf_results_df["predicted_sales"] = rf_pred

rf_daily_results = rf_results_df.groupby("date", as_index=False).agg({
    "actual_sales": "sum",
    "predicted_sales": "sum"
})

fig = px.line(
    rf_daily_results,
    x="date",
    y=["actual_sales", "predicted_sales"],
    title="Actual vs Predicted Sales - Tuned Random Forest",
    labels={
        "date": "Date",
        "value": "Sales",
        "variable": "Sales Type"
    }
)

fig.update_layout(
    hovermode="x unified"

```

```

)
fig.update_xaxes(
    rangeslider_visible=True
)
fig.show()

rf_residuals = y_test.values - rf_pred

rf_residual_df = pd.DataFrame({
    "predicted_sales": rf_pred,
    "residuals": rf_residuals
})

rf_sample_residuals = rf_residual_df.sample(
    min(5000, len(rf_residual_df)),
    random_state=42
)

fig = px.scatter(
    rf_sample_residuals,
    x="predicted_sales",
    y="residuals",
    title="Residual Plot - Tuned Random Forest",
    labels={
        "predicted_sales": "Predicted Sales",
        "residuals": "Residuals"
    },
    opacity=0.5
)

fig.add_hline(y=0, line_dash="dash")

fig.show()

```

## Model Comparison and Final Model Selection

```

model_comparison = pd.concat(
    [lr_results, rf_results, xgb_results, sarima_results],
    ignore_index=True
)

model_comparison = model_comparison.sort_values(by="RMSE")
model_comparison

```

Based on the model comparison results, XGBoost achieved the best overall performance with the lowest RMSE and MAE and the highest R2 score. Therefore, XGBoost was selected as the final sales forecasting model.

```

joblib.dump(xgb_pipeline, "xgboost_sales_forecasting_model.pkl")

```

# Milestone 4: MLOps, Deployment, and Monitoring

## MLOps Implementation

```
import mlflow
import mlflow.sklearn
import mlflow.xgboost
import joblib
```

```
xgb_pipeline = joblib.load("xgboost_sales_forecasting_model.pkl")
```

```
mlflow.set_experiment("Sales Forecasting")
```

```
with mlflow.start_run(run_name="Final_XGBoost"):
```

```
    mlflow.log_param("Model", "XGBoost")
```

```
    mlflow.log_metric("MAE", xgb_mae)
```

```
    mlflow.log_metric("MSE", xgb_mse)
```

```
    mlflow.log_metric("RMSE", xgb_rmse)
```

```
    mlflow.log_metric("R2", xgb_r2)
```

```
    mlflow.xgboost.log_model(
        xgb_pipeline.named_steps["model"],
        artifact_path="xgboost_model"
    )
```

```
print("Experiment logged successfully.")
```

```
2026/07/06 17:31:19 WARNING mlflow.utils.git_utils: Failed to import Git (the Git executable must be specified in one of the following ways:
```

- be included in your \$PATH
- be set via \$GIT\_PYTHON\_GIT\_EXECUTABLE
- explicitly set via git.refresh(<full-path-to-git-executable>)

All git commands will error until this is rectified.

This initial message can be silenced or aggravated in the future by setting the \$GIT\_PYTHON\_REFRESH environment variable. Use one of the following values:

- quiet|q|silence|s|silent|none|n|0: for no message or exception
- warn|w|warning|log|l|1: for a warning message (logging level CRITICAL, displayed by default)
- error|e|exception|raise|r|2: for a raised exception

Example:

```
export GIT_PYTHON_REFRESH=quiet
```

```
2026/07/06 17:31:19 WARNING mlflow.utils.git_utils: Failed to import Git (the Git executable must be specified in one of the following ways:
```

- be included in your \$PATH
- be set via \$GIT\_PYTHON\_GIT\_EXECUTABLE
- explicitly set via git.refresh(<full-path-to-git-executable>)

All git commands will error until this is rectified.

This initial message can be silenced or aggravated in the future by setting the \$GIT\_PYTHON\_REFRESH environment variable. Use one of the following values:

- quiet|q|silence|s|silent|none|n|0: for no message or exception
- warn|w|warning|log|l|1: for a warning message (logging level CRITICAL, displayed b
- error|e|exception|raise|r|2: for a raised exception

Example:

```
export GIT_PYTHON_REFRESH=quiet
```

2026/07/06 17:31:19 WARNING mlflow.utils.git\_utils: Failed to import Git (the Git execut

The git executable must be specified in one of the following ways:

- be included in your \$PATH
- be set via \$GIT\_PYTHON\_GIT\_EXECUTABLE
- explicitly set via git.refresh(<full-path-to-git-executable>)

All git commands will error until this is rectified.

This initial message can be silenced or aggravated in the future by setting the \$GIT\_PYTHON\_REFRESH environment variable. Use one of the following values:

- quiet|q|silence|s|silent|none|n|0: for no message or exception
- warn|w|warning|log|l|1: for a warning message (logging level CRITICAL, displayed b
- error|e|exception|raise|r|2: for a raised exception

Example:

```
export GIT_PYTHON_REFRESH=quiet
```

2026/07/06 17:31:19 WARNING mlflow.models.model: `artifact\_path` is deprecated. Please u

Experiment logged successfully.

## Model Deployment

```
import json
metrics = model_comparison.to_dict(orient='records')
with open('model_metrics.json', 'w') as f:
    json.dump(metrics, f, indent=2)
print("model_metrics.json saved")

model_metrics.json saved
```

## Model Monitoring

```
import importlib
import monitoring

importlib.reload(monitoring)

from monitoring import (
    evaluate_model,
    detect_data_drift,
    log_evaluation,
    generate_monitoring_report,
    log_to_mlflow
)

from datetime import datetime
```

```
metrics = evaluate_model(
    xgb_pipeline,
    X_test,
    y_test
)

metrics
```

```
2026-07-06 18:04:24,193 | INFO | Model Evaluation: RMSE=331.6103, MAE=96.3638, R2=0.9407
```

```
numeric_features = X_train.select_dtypes(include="number").columns.tolist()

drift_results = detect_data_drift(
    reference_data=X_train,
    new_data=X_test,
    numeric_features=numeric_features
)

drift_results
```

```
from datetime import datetime
```

```
entry = log_evaluation(
    timestamp=datetime.now().isoformat(),
    metrics=metrics,
    drift_results=drift_results
)

entry
```

```
2026-07-06 18:06:37,525 | WARNING | DRIFT: Feature 'onpromotion' shows significant distribution drift
2026-07-06 18:06:37,532 | INFO | Monitoring log saved (1 entries).
```

```
log_to_mlflow(metrics)
```

```
2026-07-06 18:15:14,660 | INFO | Monitoring metrics logged to MLflow.
```

```
report = generate_monitoring_report()

report
```

## Model Training

```
from retrain import retrain_model

new_model = retrain_model(model_df)
```

```
2026-07-06 18:24:46,787 | INFO | =====
2026-07-06 18:24:46,790 | INFO | Starting Model Retraining Pipeline
2026-07-06 18:24:46,792 | INFO | =====
2026-07-06 18:24:46,794 | INFO | [Step 1] Validating data quality...
```

```
2026-07-06 18:24:47,476 | INFO | Training samples: 2,400,710
2026-07-06 18:24:47,478 | INFO | Test samples: 600,178
2026-07-06 18:24:47,479 | INFO | [Step 2] Building preprocessing pipeline...
2026-07-06 18:24:47,481 | INFO | [Step 3] Running time-series cross-validation...
2026-07-06 18:28:05,666 | INFO | CV RMSE: 475.12 (+/- 60.20)
2026-07-06 18:28:05,667 | INFO | [Step 4] Training final model...
2026-07-06 18:29:54,098 | INFO | MAE: 96.3638
2026-07-06 18:29:54,100 | INFO | RMSE: 331.6103
2026-07-06 18:29:54,101 | INFO | R2: 0.9407
2026-07-06 18:29:54,107 | INFO | [Step 5] Comparing with current model...
2026-07-06 18:29:54,111 | INFO | Decision: New model degraded RMSE by 0.00%. Keeping c
2026-07-06 18:29:54,115 | INFO | [Step 6] Skipping model save (no improvement).
2026-07-06 18:29:54,128 | INFO | Retraining config saved.
2026-07-06 18:29:54,131 | INFO | [Step 7] Logging to MLflow...
2026-07-06 18:29:54,508 | INFO | =====
2026-07-06 18:29:54,511 | INFO | Retraining Complete: NOT DEPLOYED
2026-07-06 18:29:54,511 | INFO | =====
```